

Scalable Agentic System Design

Handling 50 → 500 → 1000+ Tools Without LLM Tool-Confusion

Context: Datazoic technical design exercise

Concrete scenario: PayPal Postman collection (50+ APIs) scaling to multi-service catalogs (500+)

Special tools: RAG Pipeline Tool · System Search Tool

1. Problem Statement

LLMs do not scale linearly with tool count. Beyond ~10–20 tools in a single prompt, typical failure modes appear:

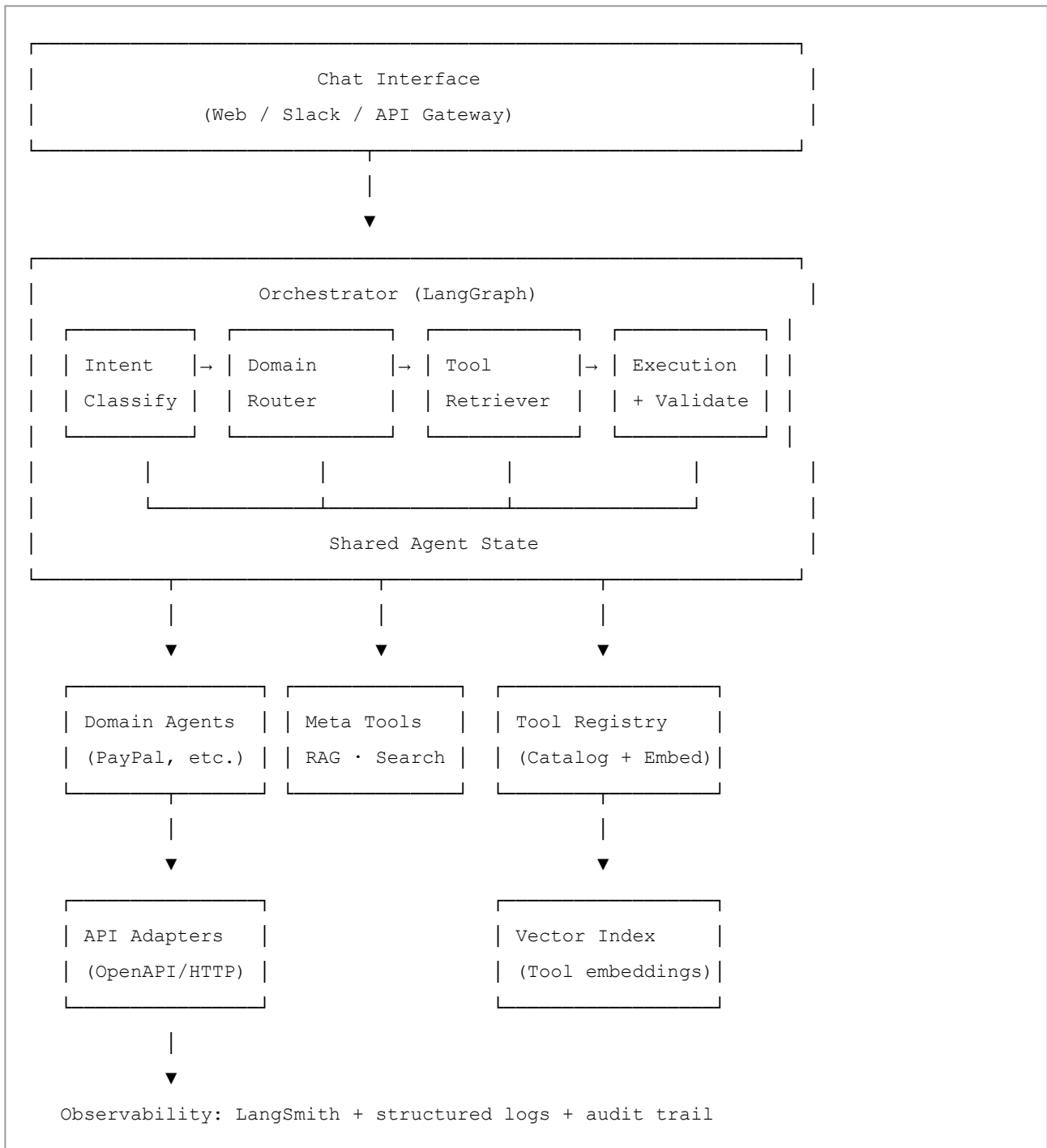
Failure Mode	Why It Happens
Wrong tool selection	Softmax over too many near-similar names/descriptions
Parameter hallucination	Schema noise + incomplete context
Premature stop / loops	Ambiguous next-step signals
Token blow-up	Full tool schemas in every turn
Latency & cost explosion	Large prompts + multi-step retries

Design goal: Keep the *decision surface* presented to any single LLM call small and relevant, while the *catalog* of capabilities grows without hard limits.

2. Design Thesis (One Sentence)

Never give the planner the full catalog. Use hierarchical retrieval + domain routing so each LLM call sees only a shortlist of 3–8 tools (or a single domain agent), then execute with schema validation, retries, and observability.

3. High-Level Architecture



Why this shape?

Flat “one agent + N tools” dies at scale. Hierarchical routing turns “pick 1 of 500” into:

1. Pick domain (≈ 5–20 options)
2. Retrieve top-k tools in that domain (≈ 3–8)
3. Plan / call with tight schemas

That keeps each LLM decision within a high-accuracy regime.

4. Framework Choice & Trade-offs

Selected stack

Layer	Choice	Why
Orchestration	LangGraph	Explicit state machine, cycles, branching, human-in-the-loop, checkpointing
Tool retrieval	LlamaIndex (or custom) + FAISS (demo) / pgvector or Pinecone (production)	Strong indexing/retrieval abstractions for tool docs & RAG KB
Tool schemas	OpenAPI → Pydantic / JSON Schema	Strict validation before HTTP
Observability	LangSmith	Traces, tool latency, failure taxonomy, eval datasets
Runtime	FastAPI + Redis (checkpoints / sessions)	Production chat + durable state

Alternatives considered

Option	Benefit	Drawback	Verdict
Plain LangChain agent	Fast to prototype	Weak control over multi-step graphs; tool overload easy	Too thin for 100+ tools
CrewAI	Nice multi-agent roles	Less precise control of routing/state; heavier opinionation	Good for demos, weaker for enterprise API ops
DSPy	Optimizable prompts/programs	Less turnkey for production workflows & HITL	Use later for prompt/tool-routing optimization
From scratch	Full control	Reinvent routing, retries, checkpoints, traces	Only if compliance forbids frameworks
AutoGen	Conversation patterns	Harder to constrain tool surfaces & costs	Skip for this use case

Why LangGraph over CrewAI: PayPal-like work is *workflow* + *correctness*, not creative roleplay. We need deterministic edges (classify → route → retrieve → act → verify), retries with typed errors, and checkpointed state for long-running invoice/dispute flows. LangGraph models that as a graph; CrewAI models it as agent chatter.

Why keep LlamaIndex in the mix: Separate concerns — LangGraph owns *control flow*; LlamaIndex (or equivalent) owns *retrieval over tools* + *docs*. Mixing both is fine when boundaries are clear.

5. Agent Structure

5.1 Roles (not infinitely many agents)

1. Supervisor / Orchestrator

Owns the conversation turn. Decides: answer directly, call RAG, call System Search, or enter a domain workflow.

2. Domain Agents (PayPal, Stripe, Reporting, ...)

Each owns a *subset* of the catalog (e.g., 30–80 tools). Still does **not** load all of them every turn — uses tool retrieval inside the domain.

3. Meta Tools (always available, small fixed set)

- `rag_pipeline` — query product/docs KB
- `system_search` — capabilities, request status, audit logs
- `ask_user` — clarify missing parameters
- `finish` — return final answer

4. Executor

Stateless function runner: validate args → call adapter → normalize response → emit structured result/error.

5.2 Conversation → Domain mapping (PayPal examples)

User utterance	Intent	Domain	Likely tools
“Send an invoice for \$50 to ...”	<code>create_invoice</code>	PayPal Invoicing	<code>create_draft_invoice</code> → <code>send_invoice</code>
“Total sales volume last month?”	<code>reporting</code>	PayPal Reporting / Transactions	<code>list_transactions</code> / reports API + aggregate
“Dispute open from user_123?”	<code>dispute_lookup</code>	PayPal Disputes	<code>list_disputes</code> / <code>get_dispute</code>

Multi-step plans are first-class: the graph can loop until `finish` or max steps.

6. Tool Selection & Routing (Core of Scale)

Stage A — Intent / Domain Classification

Lightweight LLM or classifier with **only domain labels + 1-line descriptions** (not full schemas).

```
domains = [
    paypal.invoicing,
    paypal.payments,
    paypal.disputes,
    paypal.reporting,
    meta.rag,
    meta.system_search,
    general.chat
]
```

Cost: tiny prompt. Accuracy: high because choice set is small.

Stage B — Tool Retrieval (RAG over tools)

Each tool is indexed as a document:

```
tool_id: paypal.create_draft_invoice
name: Create Draft Invoice
domain: paypal.invoicing
summary: Create a draft invoice for a recipient with amount and currency
parameters_brief: recipient_email, amount, currency, note
tags: invoice, billing, send
examples: "Send invoice for $50 to alice@acme.com"
```

Retrieve **top-k (k=3–8)** via hybrid search (BM25 + embeddings). Optionally re-rank with a cross-encoder or small LLM.

Stage C — Bound tools into the Domain Agent

Only retrieved tools (+ meta tools) are bound for that turn. The agent never sees 500 schemas.

Stage D — Optional Planner

For multi-API tasks, a short planner produces:

```
{
  "steps": [
    {"tool": "paypal.create_draft_invoice", "args_hint": {...}},
    {"tool": "paypal.send_invoice", "depends_on": 0}
  ]
}
```

Then an executor runs steps with validation.

Scaling 50 → 500 → 1000+

Catalog size	Technique
50	Domain route optional; tool retrieval alone often enough
500	Mandatory domain taxonomy + retrieval + re-rank
1000+	Multi-level taxonomy (service → capability → tool), caching of frequent shortlists, learned routers (DSPy/bandits later)

Hard rule: Max domain tools exposed per LLM call ≤ 8 (configurable), plus the 4 fixed meta-tools. Measure accuracy vs k in LangSmith evals.

7. State Management

LangGraph shared state (typed):

```
class AgentState(TypedDict):
    messages: list          # chat history (trimmed / summarized)
    user_id: str
    session_id: str
    intent: str | None
    domain: str | None
    candidate_tools: list    # retrieved shortlist
    plan: list | None
    tool_results: list       # structured outputs
    missing_slots: list      # params still needed
    status: str              # running | waiting_user | done | failed
    citations: list          # RAG sources
    audit_ids: list          # for System Search / compliance
```

Policies

- **Checkpoint** after each node (Redis / Postgres) → resume after clarification or failure.
- **Summarize** old messages when token budget tight; keep slot-filled entities (invoice_id, dispute_id) in structured fields, not only prose.
- **Idempotency keys** for mutating PayPal calls (send payment / send invoice).
- **Secrets** never in prompts — only tool runtime injects tokens from a vault.

8. The Two Required Meta-Tools

8.1 RAG Pipeline Tool

Purpose: Answer from docs/guides without guessing API behavior.

```
rag_pipeline(query: str, filters?: {product, version}) -> {  
  answer_context: str,  
  chunks: [{text, source, score}],  
  confidence: float  
}
```

Implementation sketch:

- Ingest PayPal docs, internal runbooks, error catalogs into a vector store (FAISS for the demo; pgvector or Pinecone in production) (+ BM25).
- Agent calls RAG when: user asks “how/what/why”, or before risky actions needing policy context.
- Orchestrator can also auto-call RAG when tool errors need remediation tips.

8.2 System Search Tool

Purpose: Reflective queries about the *platform itself*.

Examples:

- “What tools are available for managing invoices?”
- “What’s the status of my last request?”

```
system_search(query: str, scope: "capabilities" | "requests" | "logs") -> {...}
```

Backed by:

- **Capabilities index** = same tool registry (searchable metadata)
- **Request store** = session/run records (status, plan, tool_results)
- **Audit logs** = structured events for compliance

This prevents stuffing the entire catalog into the system prompt just to answer meta questions.

9. Execution, Validation & Error Handling

Pre-call validation

1. JSON Schema / Pydantic validate arguments
2. Slot-filling: if required fields missing → `ask_user` (do not hallucinate emails/amounts)
3. Policy checks (limits, permissions, PII)

Call adapter

- Generated from OpenAPI/Postman collection
- Timeouts, retries with exponential backoff for 429/5xx
- Map HTTP errors → `typed ToolError(code, retryable, user_message, raw)`

Recovery graph

```

success → continue plan / finish
retryable → retry (bounded)
validation_error → ask_user or re-plan with error context
not_found / business_error → RAG for guidance OR clarify
fatal → fail gracefully with audit id

```

Guardrails

- Confirm high-risk actions (send payment, refund) via HITL interrupt in LangGraph
- Dry-run mode for demos
- Max tool calls / max tokens per session

10. Scalability (Systems View)

Concern	Approach
Tool catalog growth	Registry service; CI imports Postman/OpenAPI → embeddings job
Hot paths	Cache domain→top tools for common intents
Multi-tenant	Per-tenant credentials; tool allow-lists by role
Throughput	Async workers for long reports; queue mutating ops
Horizontal scale	Stateless API + shared checkpoint store + shared vector DB (pgvector / Pinecone)
Eval at scale	Golden set of NL→tool sequences; regression in CI via LangSmith

Ingest pipeline for PayPal Postman → tools

1. Parse collection → one registry entry per request
2. Enrich with LLM-generated summary + example utterances (offline)
3. Embed & index
4. Attach auth scopes & risk tier
5. Version the catalog (tools are versioned like APIs)

11. Observability (LangSmith + more)

Instrument every node and tool:

- Trace: user message → classify → retrieve(k) → tool calls → final answer
- Metrics: tool-selection accuracy, param validity rate, avg tools-exposed, latency, \$ cost
- Datasets: labeled utterances for invoice/dispute/report
- Alerts: spike in wrong-domain routing or validation failures

Without observability, hierarchical routing is unverifiable theater.

12. End-to-End Example

User: "Send an invoice for \$50 to alice@acme.com"

1. **Classify** → `paypal.invoicing / create_invoice`
2. **Retrieve tools** → `create_draft_invoice, send_invoice, get_invoice`
3. **Plan** → create draft → send
4. **Validate** slots: amount=50, currency=USD (default/ask), recipient=alice@acme.com
5. **HITL** (optional): "Confirm send invoice \$50 to alice@acme.com?"
6. **Execute** adapters → capture `invoice_id` in state
7. **Finish** with confirmation + id
8. **Audit** written → later answerable via `system_search`

User: "What was my total sales volume last month?"

1. Route → reporting
2. Retrieve report/transaction tools
3. May call RAG for "which report endpoint is correct"
4. Fetch + aggregate (code tool or deterministic reducer — don't trust LLM math for totals)
5. Return number + period + source

User: "What tools are available for managing invoices?"

1. Supervisor calls `system_search(scope=capabilities)`
 2. Returns filtered catalog summaries — no PayPal HTTP call needed
-

13. Addressing the Hint Areas Explicitly

Agent Structure

Supervisor + domain specialists + fixed meta-tools. Prefer *few* durable agents over one mega-agent or dozens of micro-agents.

Tool Selection & Routing

Two-stage: domain classify → semantic tool retrieval → bind shortlist. Never dump full catalog into context.

State Management

Typed graph state, checkpoints, structured slots, message summarization, idempotency for side effects.

Scalability

Registry + embeddings + taxonomy + caching + async execution; accuracy governed by *tools-per-prompt*, not *tools-in-system*.

Error Handling

Schema validation first; typed errors; bounded retry; clarify vs re-plan; HITL for irreversible actions; full traces.

14. Minimal Reference Skeleton (Conceptual)

```
# Pseudocode – LangGraph-style control flow

def classify(state): ...
def route_domain(state): ...
def retrieve_tools(state): # top-k from vector index within domain
    ...
def agent_decide(state): # LLM with ONLY shortlisted tools + meta tools
    ...
def execute_tool(state): # validate → HTTP → normalize
    ...
def rag_pipeline(state): ...
def system_search(state): ...
def ask_user(state): ...
def finish(state): ...

graph = StateGraph(AgentState)
graph.add_node("classify", classify)
graph.add_node("retrieve", retrieve_tools)
graph.add_node("decide", agent_decide)
graph.add_node("execute", execute_tool)
graph.add_node("rag", rag_pipeline)
graph.add_node("sys", system_search)
# conditional edges based on decide() output
```

15. How We Would Validate the Design

1. Run the same NL queries on a **flat 50-tool agent** vs the **hierarchical design** → measure the accuracy gap
 2. LangSmith traces should show only 3–5 tools bound per turn
 3. RAG should answer policy questions; System Search should list invoice tools / last run status
 4. Failure injection (400 validation) → expect a clarifying question instead of hallucination
 5. Story for 500 APIs: add a second domain pack without changing orchestrator
-

16. Risks & Honest Limitations

- Retrieval can still miss the right tool → mitigate with hybrid search, examples, and fallback “broaden search”
 - Bad tool descriptions poison routing → invest in catalog quality (as important as the agent)
 - Multi-step financial ops need HITL + idempotency — pure autonomy is unsafe
 - LLM planners remain non-deterministic — wrap math/aggregations in code
 - Framework churn is real — keep *interfaces* (Registry, Router, Executor) owned by you
 - Extra latency and cost: the classify → route → retrieve steps add time and LLM calls before the real work starts; for simple requests a flat agent is faster
 - More complexity: LangGraph plus a retrieval library means more code, more concepts to learn, and more parts to maintain than a single-agent setup
-

17. Conclusion

A scalable agentic system for 100–1000+ tools is not “a bigger prompt.” It is:

1. **A tool registry** treated as a searchable product
2. **Hierarchical routing** that keeps each LLM decision small
3. **Strict execution** (schema, policy, retries, HITL)
4. **Meta-tools** (RAG + System Search) for knowledge and self-reflection
5. **Graph orchestration + observability** (LangGraph + LangSmith) for control and proof

That design holds from a 50-call PayPal collection to hundreds of cross-service APIs because the model’s cognitive load stays roughly constant while the catalog grows.

Prepared as a design submission for the Datazoic scalable-agentic-system exercise.

